

Desarrollo de Software - PC4

Reporte de Animales Perdidos y Alertas

Apellidos y nombres: Leon Gonzales, Paul Fran
Código: 20232652A

I. Introducción al sistema:

El sistema ReporteAnimal es una aplicación que esta orientada a la gestión de reportes de mascotas perdidas con un soporte para la búsqueda con intención y con una red de cuidadores.

El objetivo principal del sistema es que se pueda registrar avistamiento de mascotas usando la geolocalización facilitando así su posterior consulta y clasificación en base a criterios funcionales como la adopción, venta o la verificación de si se ha perdido el animal.

Se usa java junto a springboot en el backend y en el frontend se usa vite, javascript y leaflet para los mapas , junto a una buena experiencia con el formulario y el mapa.

II. Arquitectura del sistema:

Backend

Arquitectura en capas principales: Controller, Facade, Repository, Model

Se cuenta con una base de datos en PostgreSQL

Se aplican 8 patrones de diseño

Frontend

Vite + Javascript

Leaflet para el mapa y la geolocalización

TailwindCSS para los estilos

Y un router basado en hash #reportes, #buscador, #cuidadores

Comunicación:

La api rest entre el frontend y backend se comunican mediante fetch, y también por intercambio de json

Controller. Se exponen los endpoints

model: las entidades del dominio

patterns: la implementación de los patrones de diseño

repository: el acceso a los datos

config: La configuración de CORS y del facade

III. Patrones de diseño implementados:

Creacionales

1. Builder (MascotaBuilder)

Aplicación

Se usa en el proyecto ReporteAnimal para tener una creación legible del objeto Mascota (RF 1.1) dado que tiene demasiados atributos opcionales como nombre, especie, raza, urlfoto y descripción

```
J MascotaBuilder.java X
src > main > java > com > reporteanimal > patterns > builder > J MascotaBuilder.java > MascotaBuilder > especie(Especie)
1  package com.reporteanimal.patterns.builder;
2
3  import com.reporteanimal.enums.Especie;
4  import com.reporteanimal.model.Mascota;
5
6  public class MascotaBuilder {
7
8      private String nombre;
9      private Especie especie;
10     private String raza;
11     private String foto;
12     private String descripcion;
13
14     public MascotaBuilder nombre(String nombre) {
15         this.nombre = nombre;
16         return this;
17     }
18
19     public MascotaBuilder especie(Especie especie) {
20         this.especie = especie;
21         return this;
22     }
23
24     public MascotaBuilder raza(String raza) {
25         this.raza = raza;
26         return this;
27     }
28
29     public MascotaBuilder foto(String foto) {
30         this.foto = foto;
31         return this;
32     }
33
34     public MascotaBuilder descripcion(String descripcion) {
35         this.descripcion = descripcion;
36         return this;
37     }
38
39     public Mascota build() {
40         Mascota m = new Mascota();
41         m.setNombre(nombre);
42         m.setEspecie(especie);
43         m.setRaza(raza);
44         m.setFoto(foto);
45         m.setDescripcion(descripcion);
46         return m;
47     }
48 }
```

2. Factory Method (IntencionBusquedaFactory):

Aplicación:

Se usa para resolver el RF 2.2, dependiendo de la intención seleccionada (Adopción, Venta, Verificar Pérdida) se crea la estrategia de búsqueda adecuada

```
J IntencionBusquedaFactory.java X
src > main > java > com > reporteanimal > patterns > factory > J IntencionBusquedaFactory.java > IntencionBusquedaFactory
1  package com.reporteanimal.patterns.factory;
2
3  import com.reporteanimal.enums.IntencionBusqueda;
4  import com.reporteanimal.patterns.strategy.*;
5
6  public class IntencionBusquedaFactory {
7
8      public static BusquedaStrategy crear(IntencionBusqueda intencion) {
9
10         return switch (intencion) {
11
12             case ADOPCION -> new BusquedaAdopcion();
13
14             case VENTA -> new BusquedaVenta();
15
16             case VERIFICAR_PERDIDA -> new BusquedaPerdida();
17         };
18     }
19 }
```

Estructurales:

3. Observer (NotificadorAlertas):

Aplicación:

Se usa para el RF 1.4 y el RF 3.3. Los usuarios y cuidadores se suscriben a un radio geográfico, de tal manera que cuando se registre una pérdida se notificará a los observadores activos.

```
J AlertaObservable.java X
src > main > java > com > reporteanimal > patterns > observer > J AlertaObservable.java > AlertaObservable
1  package com.reporteanimal.patterns.observer;
2
3  public interface AlertaObservable {
4
5      void agregarObserver(UsuarioObserver observer);
6
7      void removerObserver(UsuarioObserver observer);
8
9      void notificar(String mensaje);
10 }
```

```

J SistemaAlertas.java x
src > main > java > com > reporteanimal > patterns > observer > J SistemaAlertas.java > Language Support for Java(TM) by Red Hat > SistemaAlertas
1  package com.reporteanimal.patterns.observer;
2
3  import com.reporteanimal.model.Ubicacion;
4
5  import java.util.ArrayList;
6  import java.util.List;
7
8  import org.springframework.stereotype.Component;
9
10 @Component
11 public class SistemaAlertas implements AlertaObservable {
12
13     private final List<UsuarioObserver> observers = new ArrayList<>();
14
15     private final Ubicacion origen;
16     private final double radioKm = 1.0;
17
18     public SistemaAlertas() {
19         this.origen = new Ubicacion(-12.0464, -77.0428); // Lima default
20     }
21
22     @Override
23     public void agregarObserver(UsuarioObserver observer) {
24         observers.add(observer);
25     }
26
27     @Override
28     public void removerObserver(UsuarioObserver observer) {
29         observers.remove(observer);
30     }
31
32     @Override
33     public void notificar(String mensaje) {
34         for (UsuarioObserver o : observers) {
35             double distancia = origen.calcularDistanciaKm(o.getUbicacion());
36             if (distancia <= radioKm) {
37                 o.recibirAlerta(mensaje);
38             }
39         }
40     }
41 }

```

```

J UsuarioObserver.java x
src > main > java > com > reporteanimal > patterns > observer > J UsuarioObserver.java > UsuarioObserver
1  package com.reporteanimal.patterns.observer;
2
3  import com.reporteanimal.model.Ubicacion;
4
5  public interface UsuarioObserver {
6
7     Ubicacion getUbicacion();
8
9     void recibirAlerta(String mensaje);
10 }

```

4. Adapter (ImagenMetadataAdapter)

Aplicación:

Se usa para el RNF 2.1 , se adapta el archivo de imagen recibido, se extrae y simulará los metadatos en un formato JSON que el motor de búsqueda entiende.

```
J ImagenMetadata.java X
src > main > java > com > reporteanimal > patterns > adapter > J ImagenMetadata.java > ImagenMetadata
1  package com.reporteanimal.patterns.adapter;
2
3  public class ImagenMetadata {
4
5      private String etiqueta;
6      private String descripcion;
7
8      public ImagenMetadata(String etiqueta, String descripcion) {
9          this.etiqueta = etiqueta;
10         this.descripcion = descripcion;
11     }
12
13     public String getEtiqueta() {
14         return etiqueta;
15     }
16
17     public String getDescripcion() {
18         return descripcion;
19     }
20 }
```

```
J ImagenMetadataAdapter.java X
src > main > java > com > reporteanimal > patterns > adapter > J ImagenMetadataAdapter.java > ImagenMetadata
1  package com.reporteanimal.patterns.adapter;
2
3  import java.util.HashMap;
4  import java.util.Map;
5
6  public class ImagenMetadataAdapter {
7
8      public Map<String, String> adaptar(ImagenMetadata metadata) {
9
10         Map<String, String> json = new HashMap<>();
11
12         json.put("etiqueta", metadata.getEtiqueta());
13         json.put("descripcion", metadata.getDescripcion());
14
15         return json;
16     }
17 }
```


5. Facade(SistemaMascotaFacade)

Aplicación:

Es una interfaz unificada para la UI, expone métodos simples como `reportarPérdida()` o `buscarMascota()` de esta manera se oculta la complejidad de que se interactúa entre diversos subsistemas como alertas, geolocalización, usuarios, entre otros.

```
SistemaMascotasFacade.java x
src > main > java > com > reporteanimal > patterns > facade > SistemaMascotasFacade.java > SistemaMascotasFacade

1 package com.reporteanimal.patterns.facade;
2
3 import com.reporteanimal.enums.IntencionBusqueda;
4 import com.reporteanimal.model.Mascota;
5 import com.reporteanimal.model.Reporte;
6 import com.reporteanimal.model.Usuario;
7 import com.reporteanimal.patterns.factory.IntencionBusquedaFactory;
8 import com.reporteanimal.patterns.strategy.BusquedaStrategy;
9 import com.reporteanimal.patterns.template.ProcesadorReporte;
10 import com.reporteanimal.repository.MascotaRepository;
11 import com.reporteanimal.repository.ReporteRepository;
12 import com.reporteanimal.repository.UsuarioRepository;
13
14 import java.util.List;
15
16 public class SistemaMascotasFacade {
17
18     private final MascotaRepository mascotaRepository;
19     private final ReporteRepository reporteRepository;
20     private final UsuarioRepository usuarioRepository;
21
22     public SistemaMascotasFacade(
23         MascotaRepository mascotaRepository,
24         ReporteRepository reporteRepository,
25         UsuarioRepository usuarioRepository
26     ) {
27         this.mascotaRepository = mascotaRepository;
28         this.reporteRepository = reporteRepository;
29         this.usuarioRepository = usuarioRepository;
30     }
31
32     public Reporte registrarReporte(ProcesadorReporte procesador, Reporte reporte) {
33         procesador.procesar(reporte);
34         return reporte;
35     }
36
37     public List<Mascota> buscarMascotas(
38         IntencionBusqueda intencion,
39         List<Mascota> mascotas,
40         String criterio
41     ) {
42         BusquedaStrategy strategy = IntencionBusquedaFactory.crear(intencion);
43         return strategy.buscar(mascotas, criterio);
44     }
45
46     public Mascota guardarMascota(Mascota mascota) {
47         return mascotaRepository.save(mascota);
48     }
49
50     public Reporte guardarReporte(Reporte reporte) {
51         return reporteRepository.save(reporte);
52     }
53
54     public List<Usuario> listarUsuarios() {
55         return usuarioRepository.findAll();
56     }
57 }
```

Comportamiento

6. Strategy (EstrategiaBusqueda)

Aplicación:

Para los RF 2.3,2.4,2.5. Define los grupos de algoritmos intecambiables que son: BusquedaAdopcionStrategy, BusquedaVentaStrategy, BusquedaPerdidaStrategy

Se tiene primero la interfaz BusquedaStrategy ahora cada uno de los algoritmos la implementará de acuerdo a lo que se quiera usar se elegirá la estrategia

```
J BusquedaStrategy.java X
src > main > java > com > reporteanimal > patterns > strategy > J BusquedaStrategy.java > BusquedaStrategy
1  package com.reporteanimal.patterns.strategy;
2
3  import com.reporteanimal.model.Mascota;
4
5  import java.util.List;
6
7  public interface BusquedaStrategy {
8
9      List<Mascota> buscar(List<Mascota> mascotas, String criterio);
10 }
```

```
J BusquedaAdopcion.java X
src > main > java > com > reporteanimal > patterns > strategy > J BusquedaAdopcion.java > BusquedaAdopcion
1  package com.reporteanimal.patterns.strategy;
2
3  import com.reporteanimal.model.Mascota;
4
5  import java.util.List;
6
7  public class BusquedaAdopcion implements BusquedaStrategy {
8
9      @Override
10     public List<Mascota> buscar(List<Mascota> mascotas, String criterio) {
11         return mascotas;
12     }
13 }
```

J BusquedaVenta.java X

src > main > java > com > reporteanimal > patterns > strategy > J BusquedaVenta.java > BusquedaVenta

```
1 package com.reporteanimal.patterns.strategy;
2
3 import com.reporteanimal.model.Mascota;
4
5 import java.util.List;
6 import java.util.stream.Collectors;
7
8 public class BusquedaVenta implements BusquedaStrategy {
9
10     @Override
11     public List<Mascota> buscar(List<Mascota> mascotas, String criterio) {
12
13         return mascotas.stream()
14             .filter(m -> m.getRaza() != null
15                 && m.getRaza().toLowerCase().contains(criterio.toLowerCase()))
16             .collect(Collectors.toList());
17     }
18 }
```

J BusquedaPerdida.java X

src > main > java > com > reporteanimal > patterns > strategy > J BusquedaPerdida.java > BusquedaPerdida

```
1 package com.reporteanimal.patterns.strategy;
2
3 import com.reporteanimal.model.Mascota;
4
5 import java.util.List;
6 import java.util.stream.Collectors;
7
8 public class BusquedaPerdida implements BusquedaStrategy {
9
10     @Override
11     public List<Mascota> buscar(List<Mascota> mascotas, String criterio) {
12
13         return mascotas.stream()
14             .filter(m -> (m.getNombre() != null
15                 && m.getNombre().toLowerCase().contains(criterio.toLowerCase()))
16                 || (m.getDescripcion() != null
17                 && m.getDescripcion().toLowerCase().contains(criterio.toLowerCase())))
18             .collect(Collectors.toList());
19     }
20 }
```


7. State(EstadoAlerta)

Aplicación:

Dado que una alerta de pérdida puede pasar por varios estados: Activa, avistada, resulta, el comportamiento del sistema cambia de acuerdo al estado

```
J EstadoActivo.java X
src > main > java > com > reporteanimal > patterns > state > J EstadoActivo.java > EstadoActivo
1 package com.reporteanimal.patterns.state;
2
3 import com.reporteanimal.model.Reporte;
4
5 public class EstadoActivo implements EstadoReporte {
6
7     @Override
8     public void siguiente(Reporte reporte) {
9         reporte.setEstadoActual(new EstadoAvistado());
10    }
11
12    @Override
13    public String nombre() {
14        return "ACTIVO";
15    }
16 }
```

```
J EstadoAvistado.java X
src > main > java > com > reporteanimal > patterns > state > J EstadoAvistado.java > EstadoAvistado
1 package com.reporteanimal.patterns.state;
2
3 import com.reporteanimal.model.Reporte;
4
5 public class EstadoAvistado implements EstadoReporte {
6
7     @Override
8     public void siguiente(Reporte reporte) {
9         reporte.setEstadoActual(new EstadoResuelto());
10    }
11
12    @Override
13    public String nombre() {
14        return "AVISTADO";
15    }
16 }
```

```
J EstadoReporte.java X
src > main > java > com > reporteanimal > patterns > state > J EstadoReporte.java > EstadoReporte
1 package com.reporteanimal.patterns.state;
2
3 import com.reporteanimal.model.Reporte;
4
5 public interface EstadoReporte {
6
7     void siguiente(Reporte reporte);
8
9     String nombre();
10 }
```

```
J EstadoResuelto.java X
src > main > java > com > reporteanimal > patterns > state > J EstadoResuelto.java > EstadoResuelto
1 package com.reporteanimal.patterns.state;
2
3 import com.reporteanimal.model.Reporte;
4
5 public class EstadoResuelto implements EstadoReporte {
6
7     @Override
8     public void siguiente(Reporte reporte) {
9     }
10
11     @Override
12     public String nombre() {
13         return "RESUELTO";
14     }
15 }
```

8. Template Method (ProcesadorReporte)

Aplicación:

Para poder unificar el flujo de registro de un reporte (RF 1.1 y RF 1.3), el flujo base viene siendo: Validar datos → Guardar en la base de datos → Lanzar notificaciones, donde el anonimato del ciudadano o del dueño se maneja en los pasos específicos para ello.

```
J ProcesadorReporte.java X
src > main > java > com > reporteanimal > patterns > template > J ProcesadorReporte.java > ProcesadorReporte
1  package com.reporteanimal.patterns.template;
2
3  import com.reporteanimal.model.Reporte;
4
5  public abstract class ProcesadorReporte {
6
7      public final void procesar(Reporte reporte) {
8          validar(reporte);
9          asignarEstadoInicial(reporte);
10         persistir(reporte);
11         notificar(reporte);
12     }
13
14     protected abstract void validar(Reporte reporte);
15
16     protected abstract void asignarEstadoInicial(Reporte reporte);
17
18     protected abstract void persistir(Reporte reporte);
19
20     protected abstract void notificar(Reporte reporte);
21 }
```

```

1 package com.reporteanimal.patterns.template;
2
3 import com.reporteanimal.model.Reporte;
4 import com.reporteanimal.patterns.observer.SistemaAlertas;
5 import com.reporteanimal.patterns.state.EstadoActivo;
6 import com.reporteanimal.repository.ReporteRepository;
7
8 import org.springframework.stereotype.Component;
9
10 @Component
11 public class ReporteAnonimo extends ProcesadorReporte {
12
13     private final ReporteRepository reporteRepository;
14     private final SistemaAlertas sistemaAlertas;
15
16     public ReporteAnonimo(ReporteRepository reporteRepository, SistemaAlertas sistemaAlertas) {
17         this.reporteRepository = reporteRepository;
18         this.sistemaAlertas = sistemaAlertas;
19     }
20
21     @Override
22     protected void validar(Reporte reporte) {
23
24         if (reporte.getUbicacion() == null) {
25             throw new IllegalArgumentException("Ubicación obligatoria");
26         }
27
28         double lat = reporte.getUbicacion().getLatitud();
29         double lon = reporte.getUbicacion().getLongitud();
30
31         if (lat == 0.0 && lon == 0.0) {
32             throw new IllegalArgumentException("Ubicación inválida: coordenadas no pueden ser (0,0)");
33         }
34
35         if (lat < -90 || lat > 90) {
36             throw new IllegalArgumentException("Latitud fuera de rango");
37         }
38
39         if (lon < -180 || lon > 180) {
40             throw new IllegalArgumentException("Longitud fuera de rango");
41         }
42     }
43
44     @Override
45     protected void asignarEstadoInicial(Reporte reporte) {
46         reporte.setEstadoActual(new EstadoActivo());
47     }
48
49     @Override
50     protected void persistir(Reporte reporte) {
51         reporte.setAnonimo(anonimo: true);
52         reporteRepository.save(reporte);
53     }
54
55     @Override
56     protected void notificar(Reporte reporte) {
57         sistemaAlertas.notificar(mensaje: "Avistamiento anónimo registrado");
58     }
59 }

```

```
J ReporteTitular.java X
src > main > java > com > reporteanimal > patterns > template > J ReporteTitular.java > Language Support for Java(TM) by Red Hat > ReporteTitular

1  package com.reporteanimal.patterns.template;
2
3  import com.reporteanimal.model.Reporte;
4  import com.reporteanimal.patterns.observer.SistemaAlertas;
5  import com.reporteanimal.patterns.state.EstadoActivo;
6  import com.reporteanimal.repository.ReporteRepository;
7  import org.springframework.stereotype.Component;
8
9  @Component
10 public class ReporteTitular extends ProcesadorReporte {
11
12     private final ReporteRepository reporteRepository;
13     private final SistemaAlertas sistemaAlertas;
14
15     public ReporteTitular(ReporteRepository reporteRepository, SistemaAlertas sistemaAlertas) {
16         this.reporteRepository = reporteRepository;
17         this.sistemaAlertas = sistemaAlertas;
18     }
19
20     @Override
21     protected void validar(Reporte reporte) {
22         if (reporte.getMascota() == null || reporte.getUbicacion() == null) {
23             throw new IllegalArgumentException();
24         }
25     }
26
27     @Override
28     protected void asignarEstadoInicial(Reporte reporte) {
29         reporte.setEstadoActual(new EstadoActivo());
30     }
31
32     @Override
33     protected void persistir(Reporte reporte) {
34         reporteRepository.save(reporte);
35     }
36
37     @Override
38     protected void notificar(Reporte reporte) {
39         sistemaAlertas.notificar(
40             "Nueva mascota perdida reportada: " +
41             reporte.getMascota().getNombre()
42         );
43     }
44 }
```


IV. Funcionamiento del sistema

1. Reportes de mascotas

Permite registrar mascotas perdidas mediante reportes con geolocalización usando Leaflet para ello

ReporteAnimal

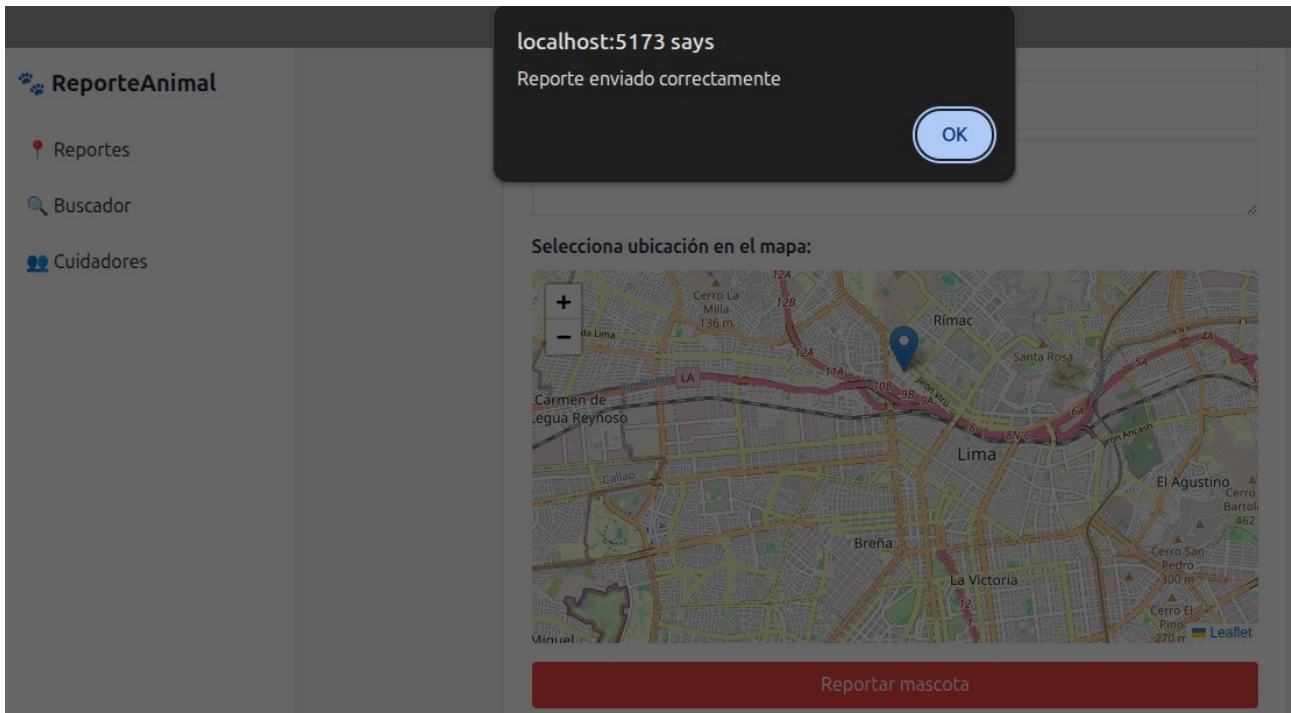
- Reportes
- Buscador
- Cuidadores

Reporte de mascota perdida

Selecciona ubicación en el mapa:



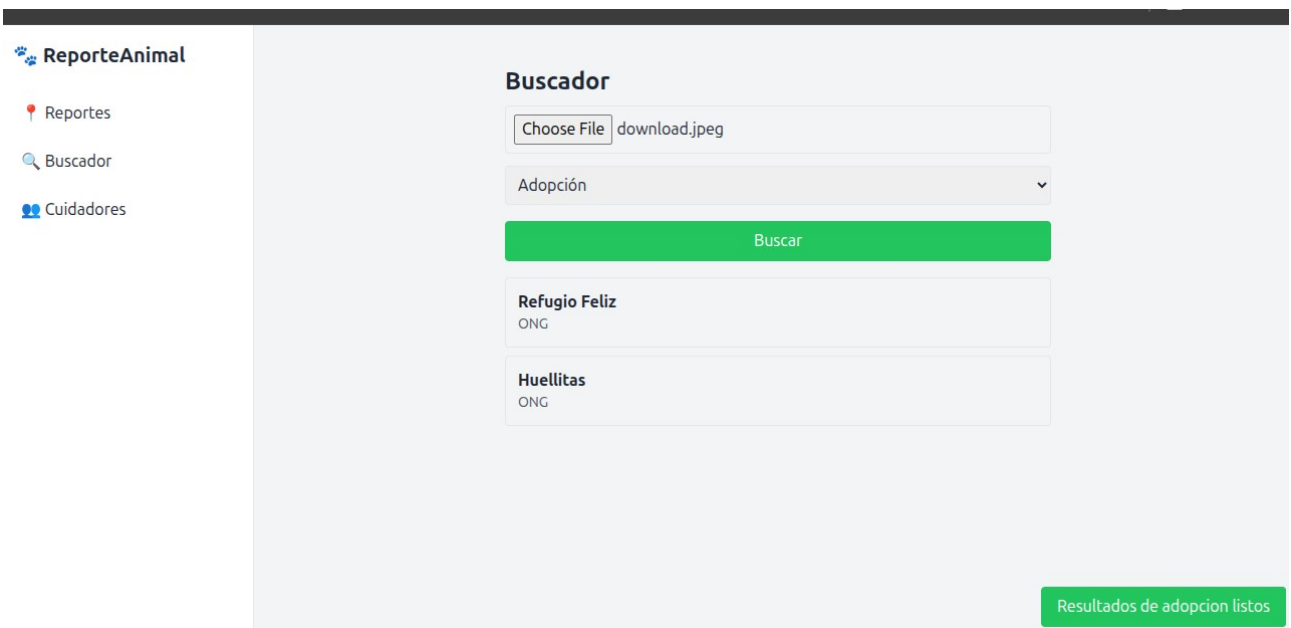
Como se ve aqui se puede elegir una posición en el mapa al dar click el marker azul indica ello



2. Buscador por intención

Se permite clasificar las consultas en, adopción (ONGs),
Venta(criaderos), Verificación(alertas)

Tambien, se puede ver que se eligio un archivo, ahi sale su nombre.



 ReporteAnimal

 Reportes

 Buscador

 Cuidadores

Buscador

Choose File download.jpeg

Venta

Buscar

Criadero Alfonso Ugarte

Certificado

Criadero en Lima Norte

Certificado

Resultados de venta listos

 ReporteAnimal

 Reportes

 Buscador

 Cuidadores

Buscador

Choose File download.jpeg

Verificar pérdida

Buscar

Sistema de alertas activas

Alerta

Resultados de verificar listos

3. Red de cuidadores

Se cuenta con una red de cuidadores que pueden recibir notificaciones de las mascotas perdidas

ReporteAnimal

- Reportes
- Buscador
- Cuidadores

Red de Cuidadores

Cuidador

Selecciona tipo

- Selecciona tipo
- Cuidador Solidario
- Profesional
- Especializado

Registrar cuidador

Se puede elegir si desea o no recibir alertas

ReporteAnimal

- Reportes
- Buscador
- Cuidadores

Red de Cuidadores

Nombre

Selecciona tipo

☐ Recibir alertas de mascotas perdidas

Registrar cuidador

Cuidador	Cuidador Solidario	Alertas: Sí
Cuidador2	Profesional	Alertas: Sí
Cuidador3	Especializado	Alertas: No

Se han implementado la mayoría de los requisitos funcionales y no funcionales de los grupos 1,2,3.

RF 1.x – Reportes, se implemento gracias al módulo de reportes y geolocalización

RF 2.x Buscador por intención

Se implemento mediante Strategy y Factory

RF 3.c Cuidadores / alertas

Se implemento con Observer y una red de cuidadores